

DTTP Trainee Management System

Training Curriculum and Design Pack

Deshmukh Technologies

Contents

DTTP Trainee Management System (TMS)	3
Document Index	3
01 — Product Requirements (PRD)	5
1. Project objective	5
2. Initial capabilities	5
3. User roles	5
4. Non-goals (early sprints)	6
5. Success criteria (Month 1 MVP)	6
6. Principles	6
02 — System Architecture	7
1. Target architecture	7
2. Request flow (learning objective)	7
3. Recommended backend layering	8
4. Frontend structure (when coding begins)	8
5. Later platform pieces	9
03 — Database Design	10
1. Database	10
2. Core tables (initial)	10
3. Trainee (Sprint 5-7)	10
4. Batches (Sprint 10)	10
5. Users & roles (Sprint 8)	11
6. Attendance (Sprint 11) — planned	11
7. Tasks / assignments (Sprint 12) — planned	11
8. Performance (Sprint 13) — planned	11
9. Design notes for trainees	11
04 — API Specification (Planned)	13
1. HTTP methods (Sprint 4)	13
2. Trainee APIs (Sprint 7 — first milestone)	13
3. Authentication (Sprint 8)	13
4. Batch APIs (Sprint 10)	14
5. Attendance (Sprint 11)	14

6. Tasks (Sprint 12)	14
7. Error responses	14
8. Tooling for API learning	15
05 — Development Roadmap	16
Phase overview	16
Phase outcomes	16
Rule	17
06 — Git and Repository Strategy	18
1. Central repository	18
2. Branch strategy	18
3. Workflow	18
4. Commit message standard	19
5. Pull request standard	19
07 — Sprint Plan	20
Sprint 0 — Environment setup	20
Sprint 1 — Python foundation	20
Sprint 2 — Virtual environment	21
Sprint 3 — FastAPI hello world	21
Sprint 4 — REST API concepts	21
Sprint 5 — MySQL	21
Sprint 6 — SQLAlchemy	22
Sprint 7 — Trainee CRUD (first real milestone)	22
Sprint 8 — Authentication	22
Sprint 9 — React frontend	22
Sprint 10 — Batch management	22
Sprint 11 — Attendance	23
Sprint 12 — Task management	23
Sprint 13 — Performance & feedback	23
After feature sprints	23
08 — Frontend Screens	24
Authentication	24
Admin	24
Mentor	24
Trainee	24
Trainee management UI flow (first assignment)	24
Dashboard metrics (later sprints)	25
UX notes for trainees	25
09 — Testing Strategy	26
1. Backend — pytest	26
2. API — Postman + Swagger	26
3. Frontend	26
4. When to introduce each layer	26
5. Quality bar for PRs	26

10 — Definition of Done	27
1. Task completion chain	27
2. Commit standard	27
3. Pull request checklist	27
4. Mentor review expectations	28
11 — 30-Day and 90-Day Plans	29
1. First 30 days	29
2. 90-day DTTP roadmap	29
3. Expansion order after MVP	30
12 — Team Structure	31
Team A — Backend	31
Team B — Frontend	31
Team C — QA	31
Team D — DevOps	31
Collaboration norms	31
13 — First Assignment: Trainee Management	32
Feature	32
Fields	32
APIs	32
Frontend	32
Acceptance criteria	32
Suggested branch	33
Suggested PR title	33
Out of scope for this assignment	33

DTTP Trainee Management System (TMS)

Deshmukh Technologies — Training Curriculum & Design Pack (Documentation Only)

Stack: React + TypeScript → FastAPI → SQLAlchemy → MySQL

Tools: GitHub · Postman · Docker · GitHub Actions

Document Index

1. 01 — Product Requirements (PRD)
2. 02 — System Architecture
3. 03 — Database Design
4. 04 — API Specification (Planned)
5. 05 — Development Roadmap
6. 06 — Git and Repository Strategy
7. 07 — Sprint Plan
8. 08 — Frontend Screens

- 9. 09 — Testing Strategy
 - 10. 10 — Definition of Done
 - 11. 11 — 30-Day and 90-Day Plans
 - 12. 12 — Team Structure
 - 13. 13 — First Assignment: Trainee Management
-

01 — Product Requirements (PRD)

1. Project objective

The **DTTP Trainee Management System (TMS)** manages the complete trainee life-cycle at Deshmukh Technologies.

Trainees build it as a production-style web application while learning:

- Python backend (FastAPI)
- React frontend (TypeScript)
- Relational database (MySQL + SQLAlchemy)
- Git / GitHub
- Testing
- Docker
- CI/CD

2. Initial capabilities

Area	Capability
Access	User login
People	Trainee registration, mentor management
Structure	Batch management
Operations	Attendance, daily tasks, assignments
Quality	Submission tracking, feedback, performance tracking
Insight	Dashboard, reports

Scope control: Not all capabilities ship in Sprint 1. Deliver a thin vertical slice first, then expand.

3. User roles

Admin

- Create trainees
- Create mentors
- Create batches
- Assign trainees to batches
- Assign mentors
- View reports
- Manage users

Mentor

- View assigned trainees
- Mark attendance
- Create tasks

- Review submissions
- Give feedback
- Track trainee progress

Trainee

- View profile
- View batch
- View attendance
- View assigned tasks
- Submit assignments
- View feedback
- View progress

4. Non-goals (early sprints)

- Full HR / payroll
- Mobile native apps
- Complex BI / data warehouse
- Multi-tenant SaaS billing
- Production CI/CD on Day 1

5. Success criteria (Month 1 MVP)

By end of Month 1 the team should have:

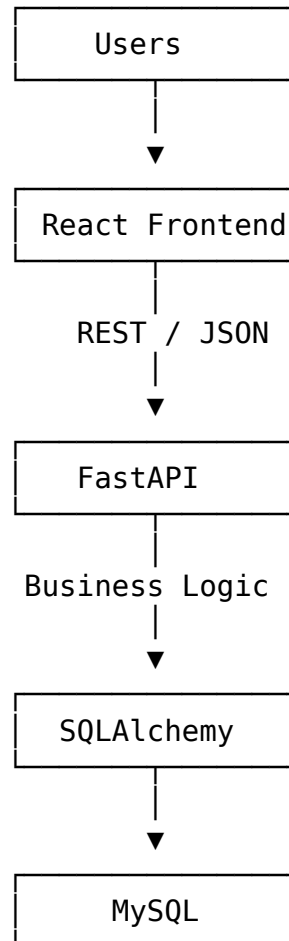
Login → Dashboard → Trainee Management → Batch Management
→ MySQL → REST APIs → React UI → GitHub PR workflow

6. Principles

1. **MVP first** — working software over complete feature list
2. **Learn by shipping** — each sprint ends in a demoable increment
3. **Mentor review required** — no direct pushes to main
4. **Definition of Done** — code + tests + docs + PR approval

02 — System Architecture

1. Target architecture



2. Request flow (learning objective)

HTTP Request
↓
FastAPI Router
↓
Python Service Function
↓
SQLAlchemy / MySQL
↓
Response (JSON)

Frontend path:
React Form
↓
POST /api/trainees

↓
FastAPI validation
↓
Service
↓
Database
↓
Response
↓
React UI update

3. Recommended backend layering

FastAPI
↓
Router
↓
Service
↓
SQLAlchemy
↓
MySQL

Suggested package layout (when coding begins):

```
backend/  
├── app/  
│   ├── main.py  
│   ├── database/  
│   │   ├── connection.py  
│   │   └── base.py  
│   ├── models/  
│   ├── schemas/  
│   ├── routers/  
│   ├── services/  
│   ├── auth/  
│   └── config/  
│       └── settings.py  
├── tests/  
├── requirements.txt  
└── .env
```

4. Frontend structure (when coding begins)

```
frontend/src/  
├── components/  
├── pages/  
└── layouts/
```

- |— services/
- |— hooks/
- |— types/
- |— utils/
- |— routes/
- |— App.tsx

5. Later platform pieces

Piece	When
Docker Compose (React + API + MySQL)	After local app works
GitHub Actions (lint, test, build)	After app is stable
Deployment / monitoring	Month 3

Do **not** introduce CI/CD on Day 1. First make the application work locally.

03 — Database Design

1. Database

```
CREATE DATABASE dttp_tms;
```

2. Core tables (initial)

Priority for early sprints: users, trainees, mentors, batches.

Later sprints add attendance, tasks, submissions, feedback, and performance scores.

3. Trainee (Sprint 5-7)

```
trainee
-----
id
name
email
mobile
batch_id
joining_date
status
created_at
updated_at
```

Status values

Status	Meaning
ACTIVE	Currently in program
INACTIVE	Temporarily inactive
COMPLETED	Finished program
DROPPED	Left program

4. Batches (Sprint 10)

```
batches
-----
id
name
start_date
end_date
status
mentor_id
```

5. Users & roles (Sprint 8)

Authentication uses a users table (or equivalent) with roles:

Role
ADMIN
MENTOR
TRAINEE

Passwords must be stored hashed (never plain text). JWT is used for API access.

6. Attendance (Sprint 11) — planned

Example fields:

Field	Notes
trainee_id	FK
date	Attendance day
status	PRESENT · ABSENT · LATE · LEAVE
marked_by	User id
remarks	Optional

7. Tasks / assignments (Sprint 12) — planned

Mentor creates tasks with title, description, deadline, priority, assignee.

Status flow:

TODO → IN_PROGRESS → SUBMITTED → REVIEW → COMPLETED

8. Performance (Sprint 13) — planned

Example scored dimensions (e.g. /10):

- Technical skills (Python, FastAPI, Database, Git)
- Problem solving
- Communication
- Task completion / code quality
- Attendance
- Professionalism

9. Design notes for trainees

- Prefer clear FK relationships over duplicated strings where possible
- Add created_at / updated_at on mutable entities

- Enforce unique emails for trainees and users
- Soft-deactivate (INACTIVE) is often safer than hard delete in early MVP

04 — API Specification (Planned)

Base path: /api

Format: JSON

Auth (from Sprint 8): Authorization: Bearer <jwt>

1. HTTP methods (Sprint 4)

Method	Use
GET	Read
POST	Create
PUT	Full update
PATCH	Partial update (optional)
DELETE	Delete / deactivate

2. Trainee APIs (Sprint 7 — first milestone)

Method	Path	Description
GET	/api/trainees	List trainees
GET	/api/trainees/{id}	Get one trainee
POST	/api/trainees	Create trainee
PUT	/api/trainees/{id}	Update trainee
DELETE	/api/trainees/{id}	Delete or deactivate

Validation rules

Field	Rule
Name	Required
Email	Valid email
Mobile	Valid format
Batch	Required (or valid batch_id)
Joining date	Valid date
Status	One of ACTIVE, INACTIVE, COMPLETED, DROPPED

3. Authentication (Sprint 8)

Flow:

Login

↓

Validate user

↓

Verify password

↓
Generate JWT
↓
Return token
↓
Access protected APIs
Protected example:
GET /api/trainees
Authorization: Bearer <token>

4. Batch APIs (Sprint 10)

Method	Path
POST	/api/batches
GET	/api/batches
GET	/api/batches/{id}
PUT	/api/batches/{id}
DELETE	/api/batches/{id}

5. Attendance (Sprint 11)

Method	Path	Notes
POST	/api/attendance	Mark attendance
GET	/api/attendance	List / filter

Reports (later): daily, monthly, trainee attendance %.

6. Tasks (Sprint 12)

Mentor creates tasks; trainee submits; mentor reviews and completes.

Exact resource paths to be finalized in the sprint issue (e.g. /api/tasks, /api/submissions).

7. Error responses

APIs should return meaningful HTTP status codes:

Code	Meaning
200	OK
201	Created
400	Bad Request
401	Unauthorized

Code	Meaning
403	Forbidden
404	Not Found
409	Conflict (e.g. duplicate email)
500	Internal Server Error

Example body:

```
{  
  "message": "Trainee not found"  
}
```

8. Tooling for API learning

- FastAPI Swagger: /docs
- Postman collections for each sprint milestone

05 — Development Roadmap

Ordered by risk reduction and learning. Each phase produces a demoable outcome.

Phase overview

Phase 1 Environment + Git
↓
Phase 2 Python + FastAPI
↓
Phase 3 MySQL + SQLAlchemy
↓
Phase 4 Trainee CRUD
↓
Phase 5 Authentication
↓
Phase 6 React Frontend
↓
Phase 7 Frontend + API Integration
↓
Phase 8 Attendance
↓
Phase 9 Tasks & Assignments
↓
Phase 10 Dashboard & Reports
↓
Phase 11 Testing
↓
Phase 12 Docker
↓
Phase 13 CI/CD
↓
Phase 14 Deployment

Phase outcomes

Phase	Outcome
1	Tools installed; repo cloned; branch workflow understood
2	FastAPI app serves / and Swagger
3	MySQL database + SQLAlchemy models connected
4	Trainee CRUD APIs work (Postman)
5	Login + JWT + role-protected routes
6	React app shell with routes
7	UI calls trainee APIs end-to-end
8	Attendance mark + list
9	Tasks, submissions, feedback flow

Phase	Outcome
10	Role dashboards + basic reports
11	pytest + Postman suites + UI smoke checks
12	docker compose up runs stack
13	GitHub Actions lint/test/build
14	Deployed environment + runbook

Rule

Do **not** jump to Docker/CI/CD before the app works locally and the MVP APIs are stable.

06 — Git and Repository Strategy

1. Central repository

Name: **dttp-trainee-management**

Recommended structure:

```
dttp-trainee-management/  
├── backend/  
├── frontend/  
├── docs/  
├── database/  
├── docker/  
├── .gitignore  
├── README.md  
└── docker-compose.yml
```

2. Branch strategy

```
main  
├── development  
│   ├── feature/authentication  
│   ├── feature/trainee-crud  
│   ├── feature/batch-management  
│   ├── feature/attendance  
│   └── feature/task-management
```

Rules

- Trainees **must not** push directly to main
- Long-lived integration branch: development
- One feature branch per issue / assignment

3. Workflow

```
Issue  
↓  
Feature Branch  
↓  
Development (local)  
↓  
Testing  
↓  
Commit  
↓  
Push
```

↓
Pull Request
↓
Mentor Review
↓
Fix Feedback
↓
Merge

4. Commit message standard

Good:

feat: add trainee CRUD API
feat: implement JWT authentication
fix: validate duplicate trainee email
test: add trainee service tests
refactor: improve trainee service

Avoid: update, changes, final, test, done

5. Pull request standard

Every PR should include:

- Title
- Description
- Changes (bullet list)
- Testing notes
- Screenshots (for UI changes)
- Related issue

Example:

feat: implement trainee CRUD

Changes:

- Added trainee model
- Added trainee schema
- Added CRUD endpoints
- Added validation

Testing:

- Tested using Postman
- Added service tests

07 — Sprint Plan

Sprint 0 — Environment setup

Objective: Every trainee can run the development environment.

Install

- Python
- VS Code
- Git + GitHub access
- MySQL + MySQL Workbench
- Postman
- Node.js
- Chrome
- Docker — later / if required

Verify

```
python --version
git --version
mysql --version
node --version
npm --version
```

Sprint 1 — Python foundation

Topics: variables, types, strings, lists/tuples/sets/dicts, conditions, loops, functions, modules, classes, exceptions, files, venv, pip.

Assignment

CLI program managing trainee records (dict-based), with:

- Add trainee
- Search trainee
- Update trainee
- Delete trainee
- Display trainees

Example record:

```
trainee = {
    "name": "Rahul",
    "email": "rahul@example.com",
    "batch": "DTTP-01",
}
```

Sprint 2 — Virtual environment

```
mkdir dttp-tms
cd dttp-tms
python -m venv venv
# Windows: .\venv\Scripts\Activate.ps1
# macOS/Linux: source venv/bin/activate
python -m pip install --upgrade pip
python --version
pip --version
```

Sprint 3 — FastAPI hello world

```
pip install fastapi uvicorn
```

```
backend/
├── app/
│   └── main.py
```

Minimal app: title DTTP Trainee Management System, GET / returns running message.

```
uvicorn app.main:app --reload
```

- App: `http://127.0.0.1:8000`
- Swagger: `http://127.0.0.1:8000/docs`

Learning objective: HTTP request → FastAPI router → Python function → response.

Sprint 4 — REST API concepts

Teach GET / POST / PUT / PATCH / DELETE with trainee-shaped routes (in-memory is OK before DB).

Sprint 5 — MySQL

- Create database `dttp_tms`
 - Design initial tables: users, trainees, mentors, batches
 - Implement trainee table fields (see [03-DATABASE-DESIGN.md](#))
-

Sprint 6 — SQLAlchemy

`pip install sqlalchemy pymysql`

Wire: FastAPI → Router → Service → SQLAlchemy → MySQL

Adopt the recommended backend package structure from [02-SYSTEM-ARCHITECTURE.md](#).

Sprint 7 — Trainee CRUD (first real milestone)

Method	Path
POST	/api/trainees
GET	/api/trainees
GET	/api/trainees/{id}
PUT	/api/trainees/{id}
DELETE	/api/trainees/{id}

Include validation + meaningful HTTP errors. Details: [13-FIRST-ASSIGNMENT-TRAINEE-CRUD.md](#).

Sprint 8 — Authentication

- Login
 - Password hashing
 - JWT
 - Authorization by role (ADMIN, MENTOR, TRAINEE)
 - Protect trainee APIs with Bearer token
-

Sprint 9 — React frontend

Create frontend/ with TypeScript React tooling. Build role-based screen shells (see [08-FRONTEND-SCREENS.md](#)).

Sprint 10 — Batch management

CRUD for batches + assign trainees/mentors as scoped in the sprint issue.

Sprint 11 — Attendance

Statuses: Present · Absent · Late · Leave

APIs: POST/GET /api/attendance

Reports: daily, monthly, attendance %.

Sprint 12 — Task management

Flow:

Mentor Creates Task



Trainee Receives Task



Trainee Works



Trainee Submits



Mentor Reviews



Feedback



Completed

Statuses: TODO → IN_PROGRESS → SUBMITTED → REVIEW → COMPLETED

Sprint 13 — Performance & feedback

Mentor scores and feedback across technical and soft-skill dimensions (see database doc).

After feature sprints

Focus	Doc
Dashboard & reports	08-FRONTEND-SCREENS.md
Testing	09-TESTING-STRATEGY.md
Docker / CI/CD / deploy	05-DEVELOPMENT-ROADMAP.md Phases 12-14

08 — Frontend Screens

Authentication

- Login

Admin

- Dashboard
- Trainees
- Mentors
- Batches
- Reports

Mentor

- Dashboard
- My Trainees
- Attendance
- Tasks
- Submissions
- Feedback

Trainee

- Dashboard
- My Profile
- Attendance
- Tasks
- Assignments
- Feedback
- Progress

Trainee management UI flow (first assignment)

Trainee List

↓

Add Trainee

↓

Edit Trainee

↓

View Details

↓

Deactivate

Dashboard metrics (later sprints)

Admin

- Total / active / completed trainees
- Batches, mentors
- Attendance summary
- Pending tasks

Mentor

- My trainees
- Today's attendance
- Pending reviews
- Assigned tasks

Trainee

- My attendance
- My tasks / pending assignments
- Completed tasks
- Performance & feedback

UX notes for trainees

- Keep forms validated client-side **and** respect API errors
- Show loading and empty states
- Mobile-friendly layout for list + form screens
- Do not over-build design systems in Sprint 9 — ship clear, usable screens

09 — Testing Strategy

1. Backend — pytest

`pip install pytest`

Cover:

- Authentication
- Trainee CRUD
- Validation
- Database operations
- Authorization (roles)

2. API — Postman + Swagger

- Maintain a Postman collection per milestone
- Use FastAPI /docs during development
- Assert status codes and error message shapes

3. Frontend

Manual / later automated checks:

- Login
- Navigation
- Forms + validation
- API error handling
- Responsive layout

4. When to introduce each layer

Layer	When
Swagger smoke	From Sprint 3
Postman CRUD suite	Sprint 7
pytest service/API tests	Sprint 7-8
Frontend checklist	Sprint 9-10
CI running tests	Phase 13 (after local stability)

5. Quality bar for PRs

A feature is not ready for mentor review without:

- Happy-path verification
- At least one validation / error-path check
- Notes in the PR describing what was tested

10 — Definition of Done

1. Task completion chain

Code
↓
Validation
↓
Testing
↓
Documentation
↓
Git Commit
↓
Pull Request
↓
Mentor Review
↓
Feedback Fixed
↓
Approved
↓
Merged

A task is **complete only** when the full chain is finished.

2. Commit standard

Use conventional, descriptive messages:

```
feat: add trainee CRUD API
feat: implement JWT authentication
fix: validate duplicate trainee email
test: add trainee service tests
refactor: improve trainee service
```

Avoid vague commits (update, changes, final, done).

3. Pull request checklist

- ☐ Title describes the change
- ☐ Description includes **Changes** and **Testing**
- ☐ Related issue linked
- ☐ Screenshots attached for UI
- ☐ No secrets committed (.env, passwords, tokens)
- ☐ Branch is up to date with development (or agreed base)

4. Mentor review expectations

Reviewers check:

- Correctness vs acceptance criteria
- Validation and error handling
- Security basics (hashing, authz)
- Readability and structure
- Tests / evidence of testing

11 — 30-Day and 90-Day Plans

1. First 30 days

Week	Focus	Output
Week 1	Python + Git + FastAPI	Basic API
Week 2	MySQL + SQLAlchemy + CRUD	Trainee API
Week 3	React + API integration	Working UI
Week 4	Authentication + testing	MVP

End of Month 1 — expected system

Login
↓
Dashboard
↓
Trainee Management
↓
Batch Management
↓
MySQL
↓
REST APIs
↓
React UI
↓
GitHub PR Workflow

2. 90-day DTTP roadmap

Month 1 — Foundation

Python · FastAPI · MySQL · SQLAlchemy · React · Git · REST API · Authentication

Month 2 — Business features

Trainees · Mentors · Batches · Attendance · Tasks · Assignments · Feedback · Performance · Dashboard

Month 3 — Professional engineering

Testing · Docker · CI/CD · Security · Logging · Documentation · Deployment · Monitoring · Code review

3. Expansion order after MVP

1. Attendance
2. Tasks & submissions
3. Feedback & performance
4. Dashboards & reports
5. Hardening (tests, Docker, CI/CD, deploy)

12 — Team Structure

For a trainee batch, divide responsibilities. Rotate later so everyone gets practical exposure.

Team A — Backend

- FastAPI
- SQLAlchemy
- MySQL
- Authentication
- APIs

Team B — Frontend

- React
- UI
- Forms
- API integration
- Dashboard

Team C — QA

- Postman
- Test cases
- Bug reports
- Regression testing

Team D — DevOps

- GitHub
- Docker
- CI/CD
- Deployment

Collaboration norms

- One issue → one feature branch → one PR
- Backend and frontend agree on API contracts before UI wiring
- QA owns a Postman collection and regression checklist per sprint
- DevOps does not block Month 1 — join fully in Month 3 platform work

13 — First Assignment: Trainee Management

Feature

Trainee Management — Admin must be able to manage trainees.

This is the first real development milestone (Sprint 7 + UI integration).

Fields

Field
Trainee ID
Name
Email
Mobile
Batch
Joining Date
Status

APIs

Method	Path
POST	/api/trainees
GET	/api/trainees
GET	/api/trainees/{id}
PUT	/api/trainees/{id}
DELETE	/api/trainees/{id}

Frontend

Trainee List

↓

Add Trainee

↓

Edit Trainee

↓

View Details

↓

Deactivate

Acceptance criteria

- ☐ API works
- ☐ Database works
- ☐ Validation works

- ☐ Error handling works
- ☐ Postman tests pass
- ☐ React UI works
- ☐ Git branch created
- ☐ Commit(s) created with clear messages
- ☐ Pull request submitted
- ☐ Mentor review completed
- ☐ Feedback fixed and merged

Suggested branch

feature/trainee-crud

Suggested PR title

feat: implement trainee CRUD

Out of scope for this assignment

- Attendance
- Tasks / submissions
- Performance scoring
- Docker / CI/CD
- Full mentor/trainee portals beyond what is needed to demo trainee CRUD